DistributedOptimizer: sincronizzare prima dell'update

Nota di design per il meeting · 22 settembre 2026

Proposta. Spostare la sincronizzazione dei gradienti dalla regola per parametro all'aggiornamento dell'intero modello. Optimisers.jl continua a gestire l'ottimizzazione. Il livello distribuito concorda quali gradienti ridurre e in quale ordine.

1. Il problema: la sincronizzazione dipende dai gradienti locali

In DDP ogni processo (**rank**) possiede una replica del modello e calcola i gradienti sul proprio batch. Prima dell'aggiornamento, i rank devono mediare i gradienti dello **stesso parametro**.

L'integrazione per parametro di Flux esegue un `allreduce!` dentro `apply!` della regola `DistributedOptimizer`. Optimisers.jl, però, non chiama `apply!` quando il gradiente è assente (`nothing` o `AbstractZero`). Questo comportamento è corretto localmente. Diventa un problema distribuito quando rank diversi attivano rami diversi del modello.

Le operazioni collettive si abbinano per ordine di chiamata, non per nome del parametro. L'esempio seguente assume buffer con dimensioni e tipi compatibili.

Chiamata	Rank 0 / Rank 1	Risultato
Prima	`trunk` / `trunk`	Media corretta.
Seconda	`a` / `b`	Media tra gradienti di parametri diversi.

Tabella 1: Due rami diversi possono produrre lo stesso numero di collettive: il calcolo prosegue, ma il risultato è errato.

Se un rank emette anche una terza collettiva senza un partner, il programma può bloccarsi. Entrambi i guasti sono stati riprodotti su CPU/MPI. Il problema non è quindi Optimisers.jl in sé, ma la posizione della comunicazione nella sua integrazione distribuita.

1.1. Perché non basta sostituire gli assenti con zeri

Completare tutti i gradienti mancanti con zeri rende uniforme la sequenza, ma perde una distinzione essenziale. Con Adam, un gradiente zero aggiorna comunque i momenti e può modificare il parametro. Un gradiente assente salta invece il passo e conserva lo stato.

La regola da preservare è globale: un parametro assente su tutti i rank salta il passo. Se è presente su almeno un rank, i rank sui quali è assente contribuiscono con zero e tutti applicano lo stesso aggiornamento.

2. Il design proposto

La sincronizzazione richiede una vista dell'intero modello prima di aggiornare qualsiasi parametro. Una callback che riceve un solo parametro non dispone delle informazioni necessarie per concordare la sequenza globale.

La proposta mantiene `DistributedOptimizer` come punto d'ingresso. `setup` restituisce un contenitore dedicato, `DistributedOptimizerState`, che associa il backend distribuito allo stato dell'ottimizzatore interno. `update` e `update!` usano questo contenitore per sincronizzare i gradienti, poi delegano l'aggiornamento a Optimisers.jl.

> **Prima:** gradienti locali → per ogni parametro presente: allreduce + update.
>
> **Proposta:** gradienti locali → presenza globale → gradienti mediati → update ordinario di Optimisers.jl.

2.1. Le fasi del protocollo

1. **Ordine comune.** Individuare i parametri addestrabili secondo il contratto di `Optimisers.setup`. Usare un ordine identico su ogni rank e trattare ogni parametro condiviso una sola volta, aggregandone i contributi locali.
2. **Presenza globale.** Un allreduce di un vettore di indicatori stabilisce quali parametri hanno un gradiente su almeno un rank.
3. **Media e aggiornamento.** Ridurre i gradienti globalmente presenti nell'ordine comune. Ricostruire i gradienti per Optimisers.jl, mantenendo assenti quelli inutilizzati ovunque, poi eseguire l'update interno.

Stato del parametro	Comportamento su ogni rank
Presente localmente	Contribuisce con il proprio gradiente e applica la media globale.
Assente qui, presente altrove	Contribuisce con zero e applica la stessa media globale.
Assente ovunque	Nessuna riduzione del gradiente e nessun passo dell'ottimizzatore.

La media divide per il **numero totale di rank**, non per il numero di rank attivi sul parametro. Per esempio, con quattro rank e un solo contributo `g`, il gradiente mediato è `g / 4`. Questa convenzione assume loss locali con lo stesso peso. Batch di dimensioni diverse richiedono una ponderazione esplicita.

2.2. Contratto e compromessi

Il protocollo presuppone repliche coerenti: stessa struttura addestrabile, stessi legami tra parametri condivisi e stati iniziali sincronizzati. Tutti i rank devono partecipare agli stessi passi di aggiornamento. La presenza globale non corregge modelli strutturalmente diversi.

La comunicazione resta fuori dalla differenziazione automatica. Non serve un nuovo algoritmo di ottimizzazione né un helper manuale nel training loop. Il costo è un contenitore di stato aggiuntivo, un allreduce di presenza per passo e copie dei gradienti in buffer dedicati. Il riuso dei buffer e le ottimizzazioni della comunicazione vengono dopo le verifiche di correttezza.

3. Evidenze e limiti

I risultati disponibili sostengono la scelta architetturale, ma non dimostrano ancora la correttezza di tutti i casi della versione in sviluppo.

Ambito	**Stato della verifica**
Problema originale	Riprodotti su CPU/MPI il blocco, la corruzione silenziosa e un caso simmetrico corretto.
Versione in sviluppo	Introdotto il contenitore di stato. Regressione sui rami condizionali e suite distribuita superate con 2 rank. Restano correzioni e verifiche di compatibilità. I risultati del prototipo non le sostituiscono.
GPU/NCCL	Il nuovo protocollo non è ancora verificato su GPU. Nessuna conclusione sulle prestazioni.

Le verifiche ancora aperte riguardano l'ordine di visita con definizioni personalizzate dei parametri addestrabili, le combinazioni di parametri condivisi e immutabili, e la gestione degli indicatori di presenza su GPU. La verifica GPU deve anche escludere accessi scalari non consentiti e trasferimenti indesiderati dei gradienti verso la CPU.

La prima versione considera solo gradienti di primo ordine e non supporta l'update con Enzyme `Duplicated`. `DistributedOptimizer` deve essere esterno alle regole che compone: può contenere un `OptimiserChain`, ma non esservi annidato. Le composizioni non supportate devono produrre un errore esplicito.

4. Possibile alternativa: modifica a Optimisers.jl

Il contenitore di stato è un punto d'integrazione accettabile in Flux? L'alternativa da valutare è un punto d'estensione pubblico in Optimisers.jl per l'intero albero dei gradienti. La mia idea era di partire con un wrapper all'interno di flux, e in caso questo fosse risultato eccessivamente macchinoso o lento allora avremmo potuto considerare un approccio più drastico modificando direttamente Optmisers.jl. Il vantaggio di questa strada è che la nuova funzionalità diventa più accessibile a tutto l'ecosistema Julia (es. Lux.jl)

> **Prossimo passo proposto.** Chiudere le verifiche CPU/MPI sulla versione in sviluppo. Solo dopo, verificare lo stesso contratto su GPU/NCCL e misurare il costo del protocollo.